

Probeklausur Komplexität und Berechenbarkeit

Hinweis: Diese Probeklausur deckt verschiedene Themen der Vorlesung ab und dient Ihnen als Übung/Vorbereitung für die Klausur. Da dies nur ein Beispiel ist, wie die Klausur aussehen könnte, gibt es keine Garantie, dass diese Probeklausur alle Themen abdeckt, die auch in der Klausur auftreten können. Darüber hinaus können einzelne Aufgaben auch schon einmal in den Übungen besprochen worden sein. In der finalen Klausur wird das sicherlich nicht der Fall sein.

Aufgabe 1. Grammatiken

Definieren Sie eine Grammatik G , die die folgende Sprache erzeugt:

$$L(G) = \{w \mid \exists n \in \mathbb{N} \ w = a^n b a^n\}$$

Wie gliedert sich G in die Chomsky-Hierarchie ein?

Lösung:

$G = (\{S\}, \{a, b\}, P, S)$ mit $P = \{S \rightarrow aSa, S \rightarrow b\}$

Typ2 Grammatik, kontextfrei

Aufgabe 2. Grammatiken

Welche der folgenden Sprachen sind regulär? Begründen Sie Ihre Antwort.

- a) $L_a = \{ww^R \mid w \in \{0,1\}^*\}$
- b) $L_b = \{a^n c^m b^n \mid n, m \geq 0\}$
- c) $L_c = \{w \in \{0,1\}^* \mid \text{Anz}(w,0) \text{ ist gerade und } \text{Anz}(w,1) \text{ ist durch 3 teilbar}\}$
- d) $L_d = \text{Menge aller } w \in \{0,1\}^*, \text{ so dass auf jede Null eine Eins folgt.}$

Lösung:

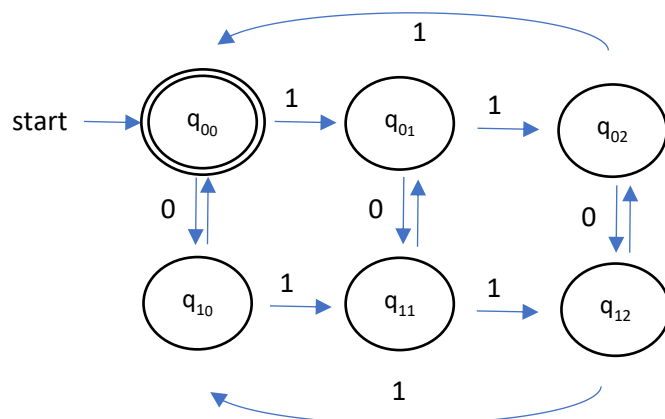
a) L_a ist kontextfrei $S \rightarrow 0S0, S \rightarrow 1S1, S \rightarrow \epsilon$ nicht regulär $\rightarrow$ Pumping-Lemma! In der Klausur müsste gezeigt werden, dass Pumping-Lemma nicht erfüllt ist.

b) L_b nicht regulär: $\rightarrow$ Pumping-Lemma!

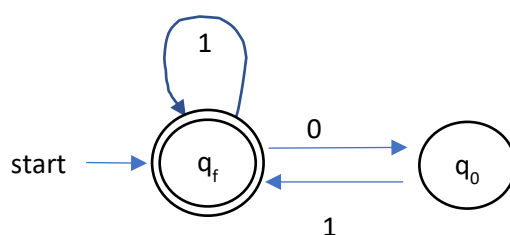
Annahme: L_b ist regulär, dann gilt das Pumping-Lemma. Es gibt ein $k \in \mathbb{N}$, sodass für $w \in L_b$ mit $|w| \geq k$ eine Zerlegung $w = uvz$ existiert sodass auch $w = uv^i z$ aus L_b .

Sei $w = a^k c^m b^k$ und $w = uvz$ die aus Pumping-Lemma existierende Zerlegung mit $|uv| \leq k$. Dann ist $w = uv^i z = a^{k+i} c^m b^k$ nicht aus L_b . Widerspruch zur Annahme, also ist L_b nicht regulär.

c) L_c ist regulär: wir können einen Automaten konstruieren. Wir benötigen zwei Zustände, um zu speichern, ob wir eine gerade Anzahl 0 gelesen haben, drei Zustände, ob Anzahl 1 durch drei teilbar ist. Also insgesamt 6 Zustände q_{ij} , mit $i = \text{anz}(0) \bmod 2$ und $j = \text{anz}(1) \bmod 3$



- d) L_d ist regulär: wir benötigen einen akzeptierenden Zustand q_f , in dem wir auch starten (das leere Wort ist in L_d). Sehen wir eine eins, so bleiben wir in diesem Zustand. Sehen wir eine 0, so gehen wir in den Zustand q_0 . Folgt anschließend eine 1, so geht der Automat wieder in den Zustand q_f .



Aufgabe 3. Turingmaschine

Geben Sie eine Turingmaschine an, welche die Sprache $L = \{a^n b^n c^m \mid n, m \geq 1\}$ akzeptiert. Machen Sie eine Laufzeitanalyse.

Lösungsidee:

Überprüfe zunächst, ob das Wort auf dem Band zunächst nur a, dann b dann c enthält ($w = a^i b^j c^k$ mit $i, j, k \geq 1$). Überschreibe danach immer wechselweise ein a und dann ein b mit Sonderzeichen. Wenn weder ein a noch ein b übrig bleibt, dann akzeptiere. Man kann Schritt 1 und Schritt 2 natürlich auch miteinander verzahnen.

Wichtig:

7-Tupel der Turingmaschine definieren!

Überföhrungsfunktion kann über eine Tabelle beschrieben werden, wie wir dies in den Übungen mehrfach getan haben.

Laufzeit $O(|w|^2)$

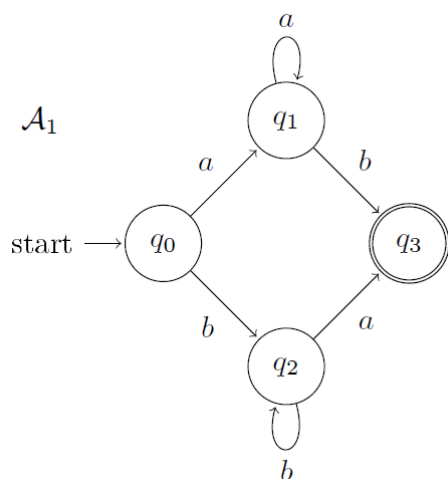
Aufgabe 4. Automaten

Geben Sie jeweils durch Transitionsgraphen einen deterministischen endlichen Automaten an, der die gegebene Sprache erkennt.

- $L_1 = \{a^n b, b^n a \text{ mit } n \geq 1\}$
- $L_2 = \{ \{a,b\}^{2n+1} \text{ mit } n \geq 0 \}$
- $L_3 = \{ab^{3n}a, \text{ mit } n \geq 1\}$

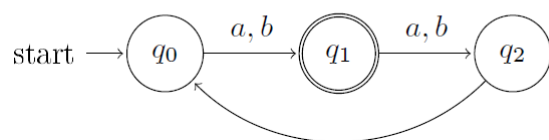
Lösung:

L1 :

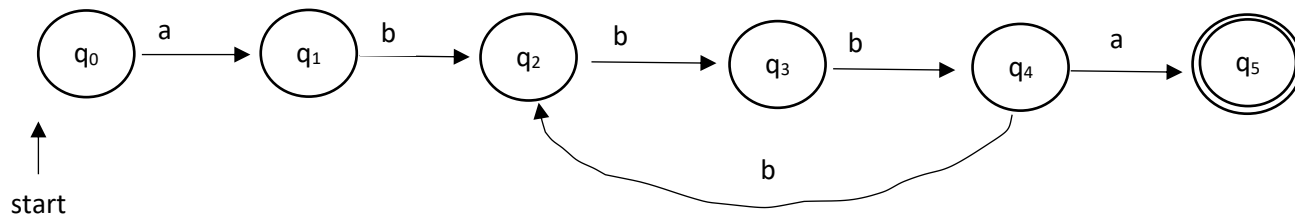


L2:

$\mathcal{A}_4$



L3



Aufgabe 5. NP-Vollständigkeit

Wir wissen, dass das Rucksackproblem Element von NP ist. Es ist sogar NP-vollständig.

PARTITION ist wie folgt definiert:

Gegeben seien k Größen $a_1, \dots, a_k \in \mathbb{N}$. Können diese in zwei gleich große Teilmengen aufgeteilt werden? D.h. es gibt eine Teilmenge $I \subseteq \{1, \dots, k\}$, sodass

$$\sum_{i \in I} a_i = \sum_{i \notin I} a_i$$

Zeigen Sie, dass PARTITION NP-vollständig ist.

Lösungsidee:

- 1) PARTITION liegt in NP
 - a. Rate eine Mengenaufteilung. Dies ist ein Wort der Länge k aus 0 und 1 (polynomieller Aufwand)
 - b. Berechne die beiden Teilsummen und überprüfe auf Gleichheit. (polynomieller Aufwand)

- 2) PARTITION ist NP-hart

Wir reduzieren das Rucksackproblem auf PARTITION.

Rucksackproblem: Gegeben k Objekte $o_1, \dots, o_k$ mit Gewichten w_i und Nutzen p_i . Gibt es eine Teilmenge I der Objekte mit $\sum_{i \in I} w_i \leq b$ und $\sum_{i \in I} p_i \geq p_0$?

Wir betrachten die Rucksack-Variante mit $w_i = p_i$, und $\sum_{i \in I} w_i = b$

Transferfunktion: Transformiere das Rucksackproblem in das PARTITION Problem. Dafür setze $a_i = w_i$, $S = \sum_i w_i$

Ergänze : $a_{k+1} = S$, $a_{k+2} = 2b$

Transferaufwand ist polynomiell. Wir müssen noch zeigen, dass eine Lösung des einen Problems existiert, genau dann wenn es eine Lösung für das andere Problem gibt.

Richtung 1: $(x \in L_1 \Rightarrow f(x) \in L_2)$

Eine Lösungsmenge I des Rucksackproblems ergibt in Summe b . Die restlichen Elemente ergeben $S-b$. Für das Partitionsproblem fügen wir zu I das Element a_{k+1} hinzu. Dies ergibt für die erste Partition die Summe $S+b$. Zur zweiten Partition fügen wir das Element a_{k+2} hinzu. Dann ergibt sich hier die Summe $S-b+2b=S+b$.

Richtung 2: ($f(x) \in L_2 \Rightarrow x \in L_1$)

Wir haben zwei gleich große Partitionen aus $a_1, \dots, a_{k+2}$. Gesamtsumme ist $S+S+2b = 2(S+b)$, Summe einer Partition ist also $S+b$

a_{k+1}, a_{k+2} können nicht beide in der gleichen Partition sein, da ihre Summe allein bereits größer als $S+b$ ist. Nehme die Partition in der $a_{k+1} = S$ enthalten ist. Die restlichen Elemente summieren dann zu b , lösen also unser Rucksack-Problem.